Moshitha K 192421205

SECTION A – DEBUGGING QUESTIONS

1. Copy Constructor Pass-by-Value Infinite Recursion Trap

Topic: Copy constructors and parameter passing

1) Compiler error: The copy-constructor parameter cannot be of the same class type passed by value. A typical diagnostic is: “invalid constructor; you probably meant ‘DynamicString (const DynamicString&)’” or “cannot copy ...”. Exact wording varies by compiler.

2) A by-value parameter itself must be initialized by copying the argument. That copy would call the same copy constructor again, which would require another copy, creating an unbounded recursive initialization. C++ therefore rejects this copy-constructor form.

3) Correct signature: DynamicString(const DynamicString& source). The reference (&) avoids making a copy just to pass the argument. const allows copying from const objects and guarantees the source is not modified.

4) Corrected complete code:

```
#include <iostream>
#include <cstring>
class DynamicString {
   char* buffer;
   int length;
public:
   DynamicString(const char* str) {
      length = std::strlen(str);
      buffer = new char[length + 1];
      std::strcpy(buffer, str);
   }
   DynamicString(const DynamicString& source) {
      length = source.length;
      buffer = new char[length + 1];
      std::strcpy(buffer, source.buffer);
   }
   ~DynamicString() { delete[] buffer; }
   void print() const { std::cout << buffer << '\n'; }
};
int main() {
   DynamicString s1("Antigravity C++");
   DynamicString s2 = s1;
   s2.print();
}
```

Output:

```
Antigravity C++
```

2. Shallow Copy and Double-Free Runtime Crash

Topic: Dynamic constructors, destructors, shallow copy and deep copy

1) The compiler-generated copy constructor copies each data member. For samples, it copies only the pointer value, not the allocated array. Thus track.samples and buf.samples contain the same address.

- Before the call: track.samples points to heap block A.
- During processAudio(track): buf.samples also points to block A; no second array is allocated.
- When processAudio ends: buf's destructor deletes block A.

- Back in main: track.samples is dangling. Accessing it is undefined behaviour; when track is destroyed, delete[] is attempted on block A again.

2) Both objects believe they own the same allocation. The first destructor frees it and the second destructor attempts to free the same memory, causing a double-free/heap-corruption error. The precise allocator message and printed addresses vary.

3) Deep-copy constructor and safe demonstration:

```
#include <iostream>
class AudioBuffer {
public:
   int* samples;
   int capacity;
   AudioBuffer(int cap) : capacity(cap), samples(new int[cap]) {
      for (int i=0; i<capacity; ++i) samples[i] = i*10;
      std::cout << "Allocated: " << samples << '\n';
   }
   AudioBuffer(const AudioBuffer& other)
      : capacity(other.capacity), samples(new int[other.capacity]) {
      for (int i=0; i<capacity; ++i) samples[i] = other.samples[i];
      std::cout << "Deep copy allocated: " << samples << '\n';
   }
   ~AudioBuffer() {
      std::cout << "Deallocating: " << samples << '\n';
      delete[] samples;
   }
};
void processAudio(AudioBuffer buf) {
   std::cout << "Processing sample 0: " << buf.samples[0] << '\n';
}
int main() {
   AudioBuffer track(5);
   processAudio(track);
   std::cout << "Back in main, sample 0: " << track.samples[0] << '\n';
}
```

4) Expected output pattern (heap addresses are different and unpredictable):

```
Allocated: 0x...A
Deep copy allocated: 0x...B
Processing sample 0: 0
Deallocating: 0x...B
Back in main, sample 0: 0
Deallocating: 0x...A
```

The two addresses must be distinct. The copy is destroyed first, then the original object is destroyed safely.

3. Stream Insertion Operator << Member Function Violation

Topic: Member versus non-member/friend operator

1) A binary member operator expression left @ right is interpreted as left.operator@(right). In std::cout << f1, the left operand is std::ostream, so the compiler looks for an ostream member operator accepting Fraction. The member declared inside Fraction is not considered in that position.

2) f1 << std::cout becomes f1.operator<<(std::cout), which matches the student's member function. It compiles but reverses the normal stream syntax and is unsuitable because users expect the stream on the left.

3–4) Correct implementation as a friend non-member function:

```
#include <iostream>
class Fraction {
   int numerator, denominator;
public:
   Fraction(int n, int d) : numerator(n), denominator(d) {}
```

```
    friend std::ostream& operator<<(std::ostream& os, const Fraction& f) {
        os << f.numerator << "/" << f.denominator;
        return os;
    }
};
int main() {
    Fraction f1(3, 4);
    std::cout << f1 << std::endl;
}
```

Output:

```
3/4
```

4. Prefix versus Postfix Increment Signature Inversion

Topic: Unary operator overloading

1) Canonical signatures: prefix: StepCounter& operator++(); postfix: StepCounter operator++(int).

2) The unused dummy int distinguishes postfix from prefix because both use the ++ token. The compiler supplies this argument for an expression such as obj++.

3) Prefix modifies the object and returns *this by reference, avoiding a copy and allowing chaining. Postfix saves the old state, increments the object, and returns the old snapshot by value.

4) Corrected code:

```
#include <iostream>
class StepCounter {
    int steps;
public:
    StepCounter(int s=0) : steps(s) {}
    StepCounter& operator++() { ++steps; return *this; }
    StepCounter operator++(int) {
        StepCounter old = *this;
        ++steps;
        return old;
    }
    void display() const { std::cout << "Steps: " << steps << '\n'; }
};
int main() {
    StepCounter c1(10);
    StepCounter c2 = ++c1;
    c2.display(); // 11
    c1.display(); // 11
    StepCounter c3 = c1++;
    c3.display(); // 11 (old value)
    c1.display(); // 12
}
```

Output:

```
Steps: 11
Steps: 11
Steps: 11
Steps: 12
```

5. Violation of Fundamental Operator Overloading Rules

Topic: Operator overloading constraints

1) The operators that cannot be overloaded are . (member access), .* (pointer-to-member access), :: (scope resolution), ?: (conditional), and sizeof. The dot operator is part of the language's built-in member-access grammar and cannot be given user-defined meaning.

2) At least one operand of an overloaded operator must be a user-defined type or enumeration. Both operands of int operator+(int, int) are built-in int values, so the declaration is illegal.

3) Programmers cannot invent operator tokens such as **. Only existing C++ operator tokens may be overloaded. Overloading cannot change precedence, associativity, or the number of operands (arity).

4) Legal replacement using == and +, with a normal power() member function:

```
#include <iostream>
class MathVector {
public:
   double x, y;
   MathVector(double a=0, double b=0) : x(a), y(b) {}
   bool operator==(const MathVector& other) const {
      return x == other.x && y == other.y;
   }
   MathVector operator+(const MathVector& other) const {
      return MathVector(x + other.x, y + other.y);
   }
   MathVector power(int p) const {
      MathVector result(1, 1);
      for (int i=0; i<p; ++i) {
         result.x *= x;
         result.y *= y;
      }
      return result;
   }
};
int main() {
   MathVector a(2,3), b(2,3);
   MathVector sum = a + b;
   std::cout << (a == b) << '\n';
   std::cout << sum.x << ", " << sum.y << '\n';
}
```

Output:

```
1
4, 6
```

Note: power() above raises each coordinate independently; it is an example replacement method, not a vector exponentiation operator.

SECTION B – PSEUDO-CODE WRITING QUESTIONS

1. Dynamic String Engine with Deep Copy and Overloaded +

Structured pseudo-code:

```
CLASS DynStr
  PRIVATE dataPtr: POINTER TO CHARACTER
          charCount: INTEGER

  CONSTRUCTOR(sourceString)
    charCount <- LENGTH(sourceString)
    dataPtr <- ALLOCATE ARRAY[charCount + 1]
    COPY sourceString INTO dataPtr

  COPY CONSTRUCTOR(REF other AS DynStr)
    charCount <- other.charCount
    dataPtr <- ALLOCATE ARRAY[charCount + 1]
    FOR i <- 0 TO charCount
      dataPtr[i] <- other.dataPtr[i]
    END FOR
```

```
  OPERATOR +(REF rhs AS DynStr) RETURNS DynStr
    total <- charCount + rhs.charCount
    temp <- ALLOCATE ARRAY[total + 1]
    COPY dataPtr INTO temp
    APPEND rhs.dataPtr TO temp
    result <- DynStr(temp)
    DEALLOCATE temp
    RETURN result

  DESTRUCTOR()
    DEALLOCATE dataPtr
    dataPtr <- NULL

  METHOD Display()
    PRINT dataPtr
END CLASS

MAIN
  strA <- DynStr("Hello ")
  strB <- DynStr("World!")
  strC <- strA + strB
  strC.Display()
END MAIN
```

Dry run: strA has 6 characters, so its buffer is 7 bytes; strB has 6 characters, so its buffer is 7 bytes. Concatenation length is 12, requiring 13 bytes including the null terminator. Characters copied are “Hello ” followed by “World!”. Result: Hello World!

2. Complex Number System with Constructor Overloading and Operators

```
CLASS Complex
  PRIVATE real, imag: REAL
  CONSTRUCTOR()
    real <- 0; imag <- 0
  CONSTRUCTOR(r)
    real <- r; imag <- 0
  CONSTRUCTOR(r, i)
    real <- r; imag <- i
  OPERATOR +(REF other) RETURNS Complex
    RETURN Complex(real + other.real, imag + other.imag)
  OPERATOR *(REF other) RETURNS Complex
    RETURN Complex(real*other.real - imag*other.imag,
                   real*other.imag + imag*other.real)
  OPERATOR ==(REF other) RETURNS BOOLEAN
    RETURN real = other.real AND imag = other.imag
  METHOD Display()
    PRINT real, " + ", imag, "i"
END CLASS
```

Product trace: $(3 + 2i)(1 + 4i) = (3\times1 - 2\times4) + (3\times4 + 2\times1)i = (3 - 8) + (12 + 2)i = -5 + 14i$.

Output: -5 + 14i

3. Financial Currency Class with Stream I/O and Unary Negation

```
CLASS Currency
  PRIVATE dollars, cents: INTEGER
  CONSTRUCTOR(d=0, c=0)
    totalCents <- d*100 + c
    dollars <- totalCents DIV 100
    cents <- ABS(totalCents MOD 100)
    IF totalCents < 0 AND cents != 0
      dollars <- dollars - 1
      cents <- 100 - cents
    END IF
  OPERATOR -() RETURNS Currency
    RETURN Currency(-(dollars*100 + cents) DIV 100,
```

```
            -(dollars*100 + cents) MOD 100)
  FRIEND OPERATOR <<(REF outStream, REF cur)
    PRINT "$", cur.dollars, ".", TWO_DIGIT(cur.cents)
  FRIEND OPERATOR >>(REF inStream, REF cur)
    READ cur.dollars, cur.cents
    NORMALIZE cur
END CLASS

MAIN
  cur <- Currency(5, 275)
  PRINT cur
  debt <- -cur
  PRINT debt
END MAIN
```

Trace: 5 dollars + 275 cents = 775 cents. Normalized value = $7.75. Unary negation gives −$7.75. The pseudo-code uses total cents to keep normalization consistent for positive and negative values.

Output: $7.75 and -$7.75.

4. Polynomial Expression with [] and Destructor

```
CLASS Polynomial
  PRIVATE degree: INTEGER
          coeffArray: POINTER TO REAL
  CONSTRUCTOR(deg)
    degree <- deg
    coeffArray <- ALLOCATE ARRAY[degree + 1]
    FOR i <- 0 TO degree
      coeffArray[i] <- 0.0
  COPY CONSTRUCTOR(REF other)
    degree <- other.degree
    coeffArray <- ALLOCATE ARRAY[degree + 1]
    FOR i <- 0 TO degree
      coeffArray[i] <- other.coeffArray[i]
  OPERATOR [](index) RETURNS REF REAL
    IF index < 0 OR index > degree
      RAISE BOUNDS ERROR
    RETURN REF coeffArray[index]
  OPERATOR +(REF other) RETURNS Polynomial
    maxDegree <- MAX(degree, other.degree)
    result <- Polynomial(maxDegree)
    FOR i <- 0 TO maxDegree
      a <- IF i <= degree THEN coeffArray[i] ELSE 0
      b <- IF i <= other.degree THEN other.coeffArray[i] ELSE 0
      result.coeffArray[i] <- a + b
    RETURN result
  DESTRUCTOR()
    DEALLOCATE coeffArray
END CLASS
```

Dry run: P1 = $3x^2 + 2x + 5$ has coefficients [5,2,3,0]. P2 = $4x^3 + x^2 + 7$ has coefficients [7,0,1,4]. Add by powers: constant 5+7=12; x coefficient 2+0=2; x^2 coefficient 3+1=4; x^3 coefficient 0+4=4. Result: $4x^3 + 4x^2 + 2x + 12$.

5. 2D Vector with Prefix/Postfix Increment and +=

```
CLASS Vector2D
  PRIVATE x, y: REAL
  CONSTRUCTOR(a=0.0, b=0.0)
    x <- a; y <- b
  OPERATOR ++() RETURNS REF Vector2D       // Prefix
    x <- x + 1; y <- y + 1
    RETURN REF THIS
  OPERATOR ++(dummy INTEGER) RETURNS Vector2D  // Postfix
    old <- COPY THIS
    x <- x + 1; y <- y + 1
    RETURN old
  OPERATOR +=(REF rhs) RETURNS REF Vector2D
```

```
  x <- x + rhs.x; y <- y + rhs.y
  RETURN REF THIS
END CLASS
```

Trace with v1 initially (5,8): v2 <- ++v1 increments first, so v1=(6,9) and v2=(6,9). Then v3 <- v1++ saves the old value (6,9) in v3 and increments v1 to (7,10). Final values: v1=(7,10), v2=(6,9), v3=(6,9).